Advanced Web Technologies

Lab1
Python + Flask + Fetch

Campus Support Mini

Lebanese University • Master M1S1 • IN432

Independent practical work after the theory block

Purpose

Build a small full-stack web application that displays fictional campus support tickets. You will connect an HTML page, browser JavaScript, HTTP requests, Flask routes and JSON responses.

Work mode

Work individually on your laptop. You may compare observations with a neighbour, but keep your own files and record your own checkpoints. The tutorial is designed for approximately two hours after the lecture and demonstrations.

The target interaction

User action	Expected application behaviour
Choose All	The API returns 3 tickets.
Choose Open	The API returns the 2 OPEN tickets.
Choose Resolved	The API returns the 1 RESOLVED ticket.
Send an unsupported status	The API returns HTTP 400 with a JSON error.

Before you start

Prerequisites

- Python 3.9 or newer. Flask 3.1 supports Python 3.9 and newer.
- A code editor, a terminal and a modern browser.
- Internet access may be needed for the first Flask installation. The application itself runs locally after installation.

Use the terminal commands for your operating system. If the command python is not available, try python3 on macOS/Linux or py on Windows. If activating the environment is blocked, use its Python interpreter directly in the commands.

Milestone plan

Milestone	Minutes	Observable result
1. Setup and first routes	15	The server opens and /api/health returns JSON.
2. Serve the interface	15	The page opens through Flask.
3. Return ticket records	20	The API returns the 3 fictional records.
4. Fetch and render	25	The page displays records from the API.
5. Filter and validate	20	Open returns 2; invalid input returns 400.
6. Observe failures	10	HTTP and network failures are explained.
7. Document and explain	15	The project and request evidence are ready.

Use the elapsed times as a working rhythm. If a core milestone takes longer, record the issue and continue with the next checkpoint when possible.

The data used throughout the tutorial

id	title	status
1	Wi-Fi unavailable	OPEN
2	Printer offline	RESOLVED
3	Projector not detected	OPEN

Milestone 1 — Setup and first routes

Time budget: 15 minutes.

Create an isolated Python environment, install Flask and start a minimal application.

Create a folder named campus-support-mini. Inside it, create a virtual environment and a file named app.py.

The terminal shows the Flask development server running. Opening / displays a heading, and /api/health returns JSON containing status equal to ok.

Useful hints

- Do not install Flask globally for this exercise. Keep the installation in .venv.
- The option --app app tells Flask to import app.py and find the application object.
- If port 5000 is busy, add --port 5001 and use the new URL in later tests.

Understanding questions

- Which process is listening for the browser request?
- Which route returns HTML, and which route returns JSON?
- What is the difference between a Flask application object and a route function?

```
mkdir campus-support-mini
cd campus-support-mini
python -m venv .venv
```

Activate the environment:

```
# Windows PowerShell
.\.venv\Scripts\Activate.ps1

# macOS or Linux
source .venv/bin/activate
python -m pip install "Flask>=3.1,<3.2"
python -m flask --version
```

Create app.py with this starting point:

```
from flask import Flask, jsonify

app = Flask(__name__)


@app.get("/")
def home():
    return "<h1>Campus Support Mini</h1>"


@app.get("/api/health")
def health():
    return jsonify(status="ok")
python -m flask --app app run --debug
```

Checkpoint

- The terminal shows a local HTTP URL.
- GET / displays Campus Support Mini.
- GET /api/health displays a JSON object with status equal to ok.

Milestone 2 — Serve the interface

Time budget: 15 minutes.

Move the visible interface into a static/index.html file and let Flask serve it at the root route.

Create a static folder and copy the HTML starter from the appendix. Replace the current home function so it returns app.send_static_file("index.html"). Do not keep two functions registered for GET /.

Opening the Flask URL shows the form, feedback paragraph and empty ticket list. The button may still say Complete milestone 4 because the API request has not been implemented yet.

Useful hints

- The file path should be static/index.html relative to app.py.
- Flask's send_static_file method returns the existing file as the root response.
- The starter includes a renderer and a form handler, but it deliberately does not call the ticket API yet.

Understanding questions

- Why must the page be opened through http://127.0.0.1:5000/ rather than by double-clicking index.html?
- Which HTML element receives the rendered ticket entries?
- Which JavaScript function is responsible for changing the list?

```
mkdir static
# Create static/index.html and copy the starter from the appendix.
```

Replace the first route in app.py with:

```
@app.get("/")
def home():
    return app.send_static_file("index.html")
```

Refresh the browser. Use the browser developer tools to inspect the page structure and resize the window once to observe the flex-wrap rule.

Checkpoint

- GET / returns the interface document.
- The page contains #filters, #status, #load, #feedback and #ticket-list.
- The browser console has no syntax error.

Milestone 3 — Return ticket records

Time budget: 20 minutes.

Make Flask return the fictional records as a JSON response.

Add TICKETS to app.py and register GET /api/tickets. Begin with the route returning all records. Keep the response shape {"items": [...], "count": number}.

Opening /api/tickets in the browser returns JSON containing three records and count equal to 3. The page still does not display them through Fetch until milestone 4.

Useful hints

- Use a list of dictionaries, one dictionary per ticket.
- Return jsonify(items=TICKETS, count=len(TICKETS)).
- Reload the server if debug mode does not detect the change.

Understanding questions

- Why is the data currently held in Python memory?
- What does jsonify change about the response representation?
- What is the difference between an item in the list and the count field?

```
TICKETS = [
  {"id": 1, "title": "Wi-Fi unavailable", "status": "OPEN"},
  {"id": 2, "title": "Printer offline", "status": "RESOLVED"},
  {"id": 3, "title": "Projector not detected", "status": "OPEN"},
]


@app.get("/api/tickets")
def list_tickets():
  return jsonify(items=TICKETS, count=len(TICKETS))
```

Checkpoint

- GET /api/tickets returns 3 records.
- The response has items and count keys.
- The response Content-Type is application/json.

Milestone 4 — Fetch and render

Time budget: 25 minutes.

Connect the browser to the API and display the records returned by Flask.

In loadTickets, replace the intentional placeholder inside the try block with an asynchronous Fetch request. Check response.ok, parse response.json(), call renderTickets(data.items) and display data.count in the feedback paragraph.

Submitting the form displays the three records returned by the API. The browser Network panel shows GET /api/tickets with a successful response.

Useful hints

- Keep the provided try, catch and finally structure.
- Fetch produces a Response. response.json() reads and parses its body.
- Use response.ok before parsing so an unsuccessful response is handled as an error by your code.
- The renderer should receive data.items, not the whole response object.

Understanding questions

- Where does loadTickets execute?
- Which line sends the HTTP request?
- What is the difference between the Response object and the parsed data object?
- Why should the old list be cleared when loading begins?

The starter's loadTickets function contains a clear loading state and a TODO region. Complete the logic inside try. Keep the form submit handler and the renderer.

```
const response = await fetch("/api/tickets", {
  headers: { Accept: "application/json" }
});
if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}
// TODO: parse the JSON body into data.
// TODO: render data.items.
// TODO: display data.count in feedback.
```

Test this milestone with the browser and Network panel. A successful request should show status 200 and application/json.

Checkpoint

- The form submission does not navigate away from the page.
- Three ticket titles appear in the list.
- The feedback text reports that 3 tickets were loaded.
- The button is enabled again after completion.

Milestone 5 — Filter and validate input

Time budget: 20 minutes.

Use the selected status to create a query parameter and apply the rule on the backend.

Read request.args.get("status") in Flask. Treat an absent parameter as All. Accept OPEN and RESOLVED. Return HTTP 400 JSON for an unsupported supplied value, including an explicitly supplied empty status. In the browser, append ?status=... only when a status is selected.

All returns 3, Open returns 2, Resolved returns 1. An unsupported request returns 400 and a JSON error. A valid filter with no matches returns 200 with items: [] and count: 0.

Useful hints

- Use status is not None to distinguish an absent parameter from a supplied empty value.
- For a valid status, use a list comprehension to select matching dictionaries.
- For All, omit the query parameter instead of sending status=.
- Keep the same JSON shape for successful filtered responses.

Understanding questions

- What value does request.args.get return when the parameter is absent?
- Why is the backend validation needed even when the dropdown has only two choices?
- Which part of the URL changes when the selection changes?
- Why should encodeURIComponent be used when building a query value?

Extend list_tickets with the validation and filtering logic. Place it after status is read and before the final jsonify response.

```
status = request.args.get("status")
if status is not None and status not in {"OPEN", "RESOLVED"}:
    return jsonify(error="Unsupported status"), 400

items = TICKETS
if status is not None:
    items = [t for t in TICKETS if t["status"] == status]
return jsonify(items=items, count=len(items))
```

In loadTickets, read select.value. Build query as an empty string for All and as ?status= plus an encoded value for the other choices.

```
const status = select.value;
const query = status
  ? "?status=" + encodeURIComponent(status) : "";
const response = await fetch("/api/tickets" + query, {
  headers: { Accept: "application/json" }
});
```

Checkpoint

- All displays 3; Open displays 2; Resolved displays 1.
- GET /api/tickets?status=UNKNOWN returns 400 JSON.
- The Network panel shows the query parameter for Open and Resolved.
- The page displays a useful error message when the request is unsuccessful.

Milestone 6 — Observe failures and interface states

Time budget: 10 minutes.

Use controlled failures to explain the difference between an HTTP error response and a connection failure.

Record a small request matrix. Test a missing path and a wrong method. Then test a connection failure using the already loaded page and a stopped local server or browser offline mode. Restore the working URL and server afterwards.

You can explain what the browser receives for 404 and 405, and why a network failure may happen before a Response exists. The button is enabled again because finally runs after either outcome.

Useful hints

- Use curl -i or the browser Network panel to see status and Content-Type.
- On Windows, use curl.exe if curl resolves to another command.
- For the connection test, do not reload a page that needs the server before you have restored the server.

Understanding questions

- What status does GET /api/missing produce?
- What status does POST /api/tickets produce in this exercise?
- Does Fetch automatically throw for an HTTP 404?
- What interface state should finally restore?

Request	Expected observation
GET /	200, text/html
GET /api/health	200, application/json
GET /api/tickets?status=OPEN	200, application/json, count 2
GET /api/tickets?status=UNKNOWN	400, application/json
GET /api/missing	404, usually HTML from Flask
POST /api/tickets	405, usually HTML from Flask

```
curl -i http://127.0.0.1:5000/api/tickets?status=OPEN
curl -i http://127.0.0.1:5000/api/missing
curl -i -X POST http://127.0.0.1:5000/api/tickets
```

Connection-failure experiment

Keep the page loaded. Stop the Flask process with Ctrl+C, submit the form again and observe the browser message. Restart the server and restore the working application. If you use browser offline mode, turn it off afterwards. The exact browser error text can vary.

Checkpoint

- You have recorded status, Content-Type and one body observation for the request matrix.
- You can distinguish an HTTP response with an error status from a failure to obtain a response.
- The application returns to a usable state after the experiment.

Milestone 7 — Document and explain

Time budget: 15 minutes.

Leave a small project that another student can run and understand.

Create README.md with setup, run and request instructions. Record the route table and the full interaction trace. Keep app.py and static/index.html in the project. Do not include .venv in a submission or shared archive.

A reader can create the environment, start the server, open the page and test the API. You can explain where the browser, Flask route and JSON response appear in the trace.

Useful hints

- Copy observed paths and statuses from your own Network panel.
- Mention that the tutorial keeps records in Python memory.
- Include the exact URL used when you run on port 5001.

Understanding questions

- What request is created when Open is selected?
- Where is the status validated?
- Where is the JSON parsed?
- What would a database add to this application?

README.md should contain at least these headings:

- Setup
- Run the application
- Routes and expected responses
- Interaction trace for Open
- Known limitation: records are in memory

```
# Campus Support Mini
## Setup
Create and activate a virtual environment, then install Flask.

## Run the application
python -m flask --app app run --debug
Open http://127.0.0.1:5000/

## Routes and expected responses
GET /                  -> HTML interface
GET /api/health        -> 200 JSON
GET /api/tickets       -> 200 JSON, 3 records
GET /api/tickets?status=OPEN -> 200 JSON, 2 records
GET /api/missing       -> 404

## Interaction trace for Open
Form submit -> Fetch -> Flask validation -> JSON -> DOM rendering

## Known limitation
Ticket records are stored in Python memory.
```

Checkpoint

- README.md explains setup and run.
- The route table matches your observed application.
- The Open trace names the browser, network and backend stages.
- The project folder contains no .venv directory in the files you share.

Completion checklist

- The application opens through a Flask URL.
- The health endpoint returns JSON.
- The API returns all three records and the selected filters.
- The backend rejects an unsupported status with HTTP 400 JSON.
- The browser checks response.ok, parses JSON and renders the DOM.
- Loading, success and failure feedback are visible.
- The button becomes usable after success and failure.
- The request matrix and README are complete.

Troubleshooting

Message or symptom	Check
No module named flask	Activate .venv or call its Python interpreter directly; then install Flask.
Address already in use	Run with --port 5001 and use that port in the browser and README.
404 for /	Confirm app.py is running and the root route is registered once.
404 for /api/tickets	Check the path spelling and the route decorator.
405 for a GET request	Check that the browser URL is correct and that the route is decorated with @app.get.
The list duplicates entries	Call list.replaceChildren() before appending new entries.
The browser shows a JSON parse error on 404	Check response.ok before response.json().
Fetch fails after opening index.html directly	Open the page through Flask so the relative API URL has the correct origin.

Optional extensions after the core work

- Add GET /api/tickets/<int:ticket_id> and return 404 JSON when the id is absent.
- Add a search query that matches words in a ticket title.
- Split the inline CSS and JavaScript into separate static files.
- Add a small test for the health route and the invalid filter.

Complete the seven core milestones before attempting an extension.

Appendix — HTML starter for milestone 2

Create static/index.html by copying the following complete starter. It includes the interface, the CSS and a renderer. The request logic in loadTickets is intentionally incomplete for milestone 4.

```
<!doctype html>
<html lang="en">
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Campus Support Mini</title>
 <style>
  * { box-sizing: border-box; }
  body { font-family: system-ui, sans-serif; font-size: 20px; margin: 0; }
  main { max-width: 42rem; margin: auto; padding: 1.5rem; }
  form { display: flex; flex-wrap: wrap; gap: 0.75rem; }
  select, button { font: inherit; padding: 0.5rem; }
  li { padding-block: 0.4rem; }
  button:disabled { opacity: 0.6; }
 </style>
</head>
<body>
 <main>
  <h1>Campus Support Mini</h1>
  <p>Fictional tickets for the IN432 workshop.</p>
  <form id="filters">
   <label for="status">Ticket status</label>
   <select id="status" name="status">
    <option value="">All statuses</option>
    <option value="OPEN">Open</option>
    <option value="RESOLVED">Resolved</option>
   </select>
   <button id="load" type="submit">Complete milestone 4</button>
  </form>
  <p id="feedback" role="status" aria-live="polite">
   Complete milestone 4 to load the records.
  </p>
  <ul id="ticket-list"></ul>
 </main>
 <script>
  const form = document.querySelector("#filters");
  const select = document.querySelector("#status");
  const button = document.querySelector("#load");
  const list = document.querySelector("#ticket-list");
  const feedback = document.querySelector("#feedback");
  function renderTickets(items) {
   list.replaceChildren();
   for (const ticket of items) {
    const li = document.createElement("li");
    li.textContent = `${ticket.title} (${ticket.status})`;
    list.append(li);
   }
  }
  async function loadTickets() {
   button.disabled = true;
   list.replaceChildren();
   feedback.textContent = "Loading...";
   try {
    // Milestone 4: add Fetch, status checking, JSON parsing and rendering.
    feedback.textContent = "Complete milestone 4.";
   } catch (error) {
    feedback.textContent = `Could not load tickets: ${error.message}`;
   } finally {
    button.disabled = false;
   }
  }
  form.addEventListener("submit", event => {
   event.preventDefault();
   loadTickets();
  });
 </script>
</body></html>
```